# Knowledge and Reasoning – Logical Languages

Artificial Intelligence

Giorgio Fumera
giorgio.fumera@unica.it

Department of Electrical and Electronic Engineering




UNICA

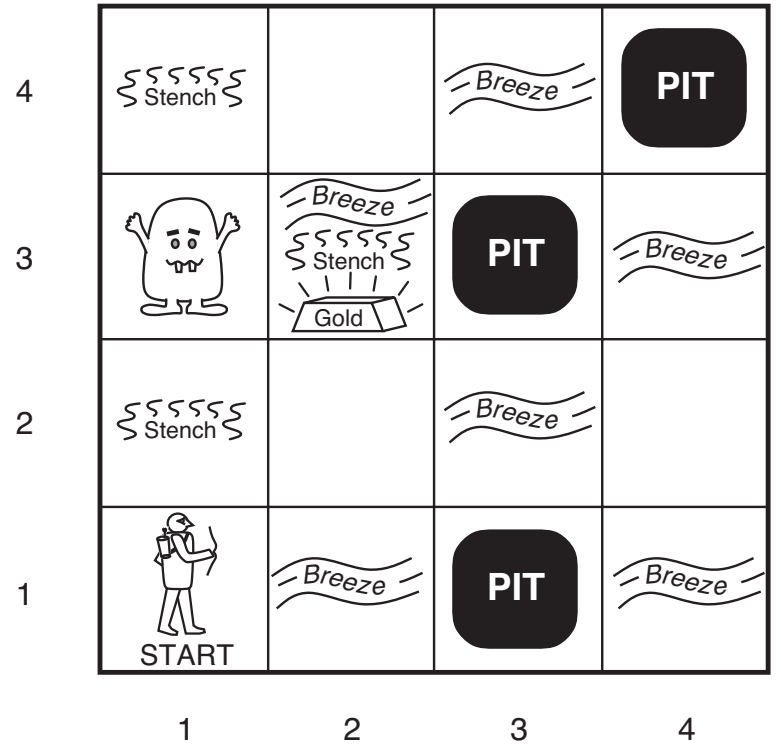
UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI

# Application examples

- Mathematics: automated theorem proving
- Logical programming languages
  - Prolog (1970s): symbolic computation (derivation, integration, etc.), parsers (compilers), etc.
  - Datalog (1970s, derived from Prolog): knowledge representation in logic-based AI systems, query language for relational databases, program analysis, etc.
- Integrated circuit design (e.g., routing FPGAs) and verification
- Software analysis and verification
- Planning (e.g., robotics)
- Computer-aided design (CAD) and manufacturing
- Scheduling (e.g.: lessons timetable, calendar of campionship matches)
- ...

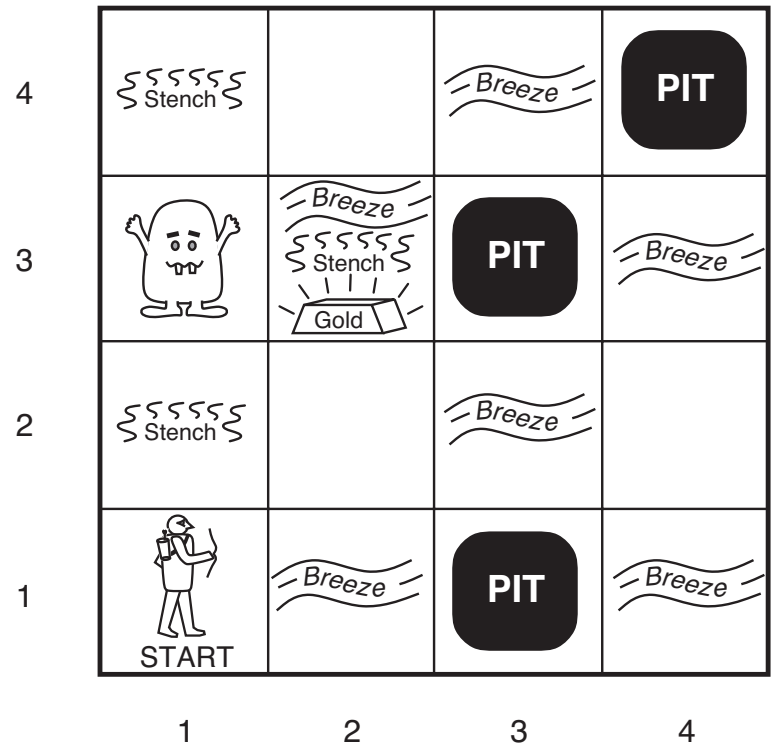
# Example: the wumpus world

- A 1975 computer game, later used as an agent testbed
- Simplified version of the game
  - a $4 \times 4$ grid of rooms containing a heap of gold, bottomless pits, and the wumpus, a monster that eats anyone entering its room
  - each room but [1,1] can contain a pit with a probability of 0.2; the wumpus and the gold are located in randomly chosen rooms, except [1,1], with a uniform distribution
  - rooms neighbouring pits are breezy
  - rooms neighbouring the wumpus are smelly

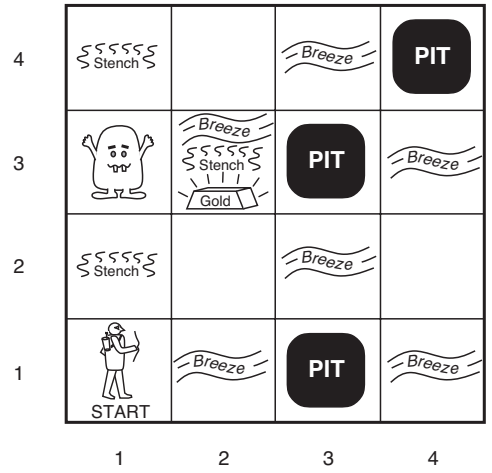


# Example: the wumpus world

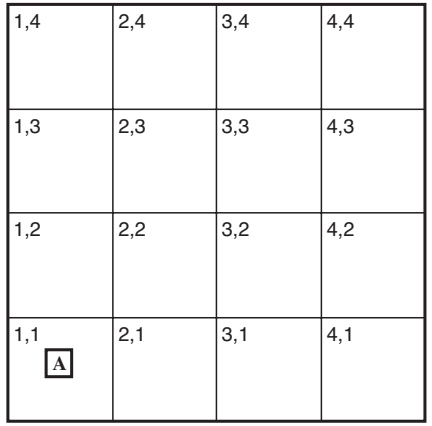
- The agent task
  - initial location: room [1,1]
  - goal: reaching the gold and going back to [1,1] without hitting the wumpus or getting trapped into a pit
  - available actions: moving to one of the adjacent rooms
  - agent's knowledge: the rules of the game, its location, and the percepts in the **visited** rooms (it does **not** know in advance the locations of the wumpus, the pits and the gold)
- How can the agent find a safe way to the gold and back to [1,1]?



# Reasoning in the wumpus world



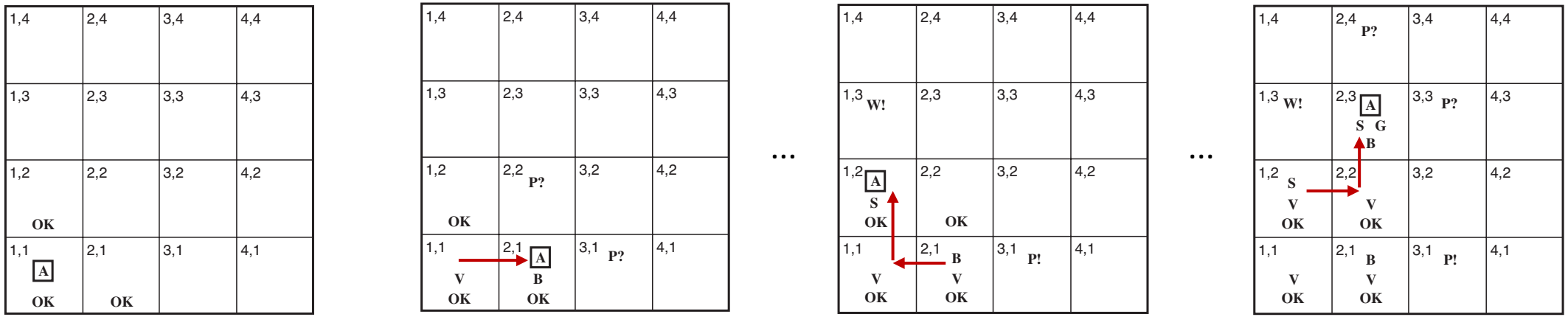

problem instance




initial knowledge of the agent

legend

| | |
|---|---|
| A | = Agent |
| B | = Breeze |
| G | = Glitter, Gold |
| OK | = Safe square |
| P | = Pit |
| S | = Stench |
| V | = Visited |
| W | = Wumpus |




sketch of the agent's reasoning and decision-making process in the first steps of the game

# Procedural vs declarative approaches

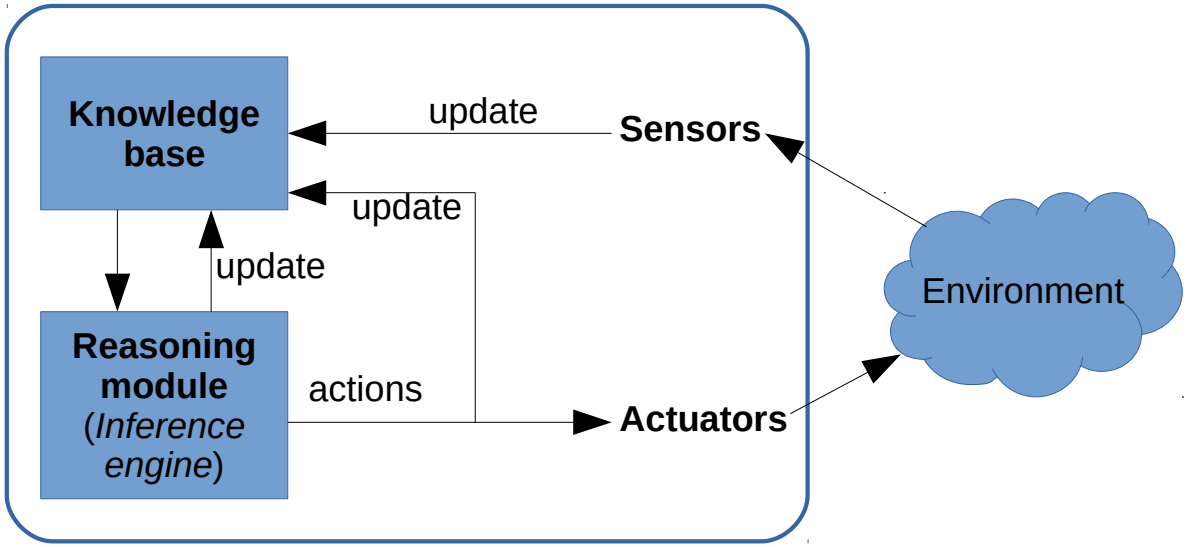
- Procedural approach
  - encoding desired behaviours directly as program code
  - example: search algorithms
- Declarative approach
  - explicit representation of problem knowledge and goal
  - behaviour determined by reasoning (inference) processes

# Knowledge-based systems (KBSs)

- KBSs follow the declarative approach
- Two distinct components: *knowledge base* and *inference module*
- Knowledge base (KB): a set of sentences in a knowledge representation language, encoding
  - **background** knowledge about a problem (e.g., rules of the wumpus world)
  - **working** knowledge about a specific problem instance
- Inference module: inference algorithms that derive new sentences "following" from the ones in the current KB

# Knowledge-based agent

- Percepts coming from sensors are encoded as sentences and added to the KB
- The inference module
  - updates the KB based on newly added sentences (percepts)
  - determines the next action



# Logic

- Origins: ancient Greek philosophy (Plato, Aristotle, etc.)
- Reducing logical inference to a mechanical process:
  W. Leibniz (1646–1716), G. Boole (1815–1864) and others
- Logic provides two main tools for KBS implementation
  - formal languages for knowledge representation
  - inference procedures

# What is *logic*?

- Discipline that studies the conditions under which *deductive argumentations* are valid (correct)
- Deductive argumentation
  - a set of **premises** (**axioms**), and a **conclusion** that should **follow** from them
  - example: *all men are mortal, Socrates is a man; Socrates is mortal*
- **Classical** logic deals with argumentations made up of **propositions** (statements) that can be either **true** or **false**

# Validity vs truth

- Examples of argumentations
  - **premises** (axioms): all men are mortal, Socrates is a man
    **conclusion**: Socrates is mortal
  - **premises** (axioms): all odd natural numbers are prime, nine is an odd natural number
    **conclusion**: nine is prime
- Propositions can be either **true** or **false**
  - their truth value depends on whether what they state is the case in the real world
- Argumentations can be either **valid** (correct) or not
  - their validity depends only on their **structure**
  - example of a valid argumentation (known as **syllogism**):
    **premises** (axioms): all *A* are *B*; *C* is *A*
    **conclusion**: *C* is *B*
  - both the above argumentations have the same structure, and are valid

# Knowledge representation

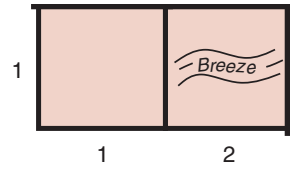
- KBs consist of sentences expressed in a **formal** language
- Syntax: specifies which sentences are **well formed**
  - example: in **arithmetic** notation "$x + y = 4$" is well formed, "$x4y +=$" is not
- Semantics: defines the **meaning** of well-formed sentences
  - meaning: **truth** value in each "possible world" (agent's environment)
  - example: "$x + y = 4$" is true if $x$ is 2 and $y$ is 2, but false if $x$ is 1 and $y$ is 1
- Models: mathematical abstraction of "possible worlds," that define a specific truth value for each relevant sentence
  - example: each possible assignment of natural numbers to $x$ and $y$ is a model that determines the truth value of any sentence of arithmetic whose variables are $x$ and $y$, such as "$x + y = 4$"

# Logical inference

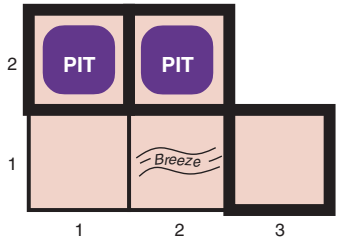
- If a sentence $\alpha$ is true in model $m$, $m$ is said to satisfy $\alpha$, or to be a model of $\alpha$
- $M(\alpha)$: the set of all models of a sentence $\alpha$
- Key concept: logical entailment between sentences
  - formalises the conditions under which a sentence $\beta$ **logically follows** from a sentence $\alpha$, or $\alpha$ **entails** $\beta$, represented as $\alpha \models \beta$:
    $\alpha \models \beta$ if and only if $\beta$ is true in every model in which $\alpha$ is true,
    i.e., if and ony if $M(\alpha) \subseteq M(\beta)$
  - example: in arithmetics $x = 0$ entails $xy = 0$, whatever the value of $y$

# Example

- Assume the agent in the wumpus world has detected nothing in [1,1] and a breeze in [2,1]
- Its KB now contains
  - the background knowledge on the wumpus world
  - the two percepts above

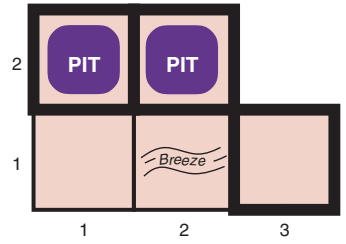



- Now it is interested in whether squares [1,2], [2,2], [3,1] contain pits
  - each square may or may not contain a pit: there are $2^3 = 8$ possible models
  - e.g., one model is "there is a pit in [1,2] and [2,2], and no pit in [3,1]"

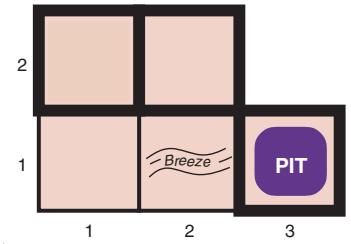


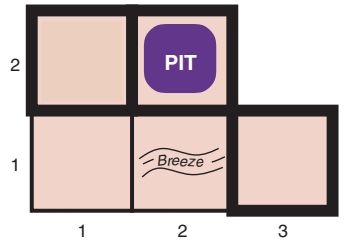
# Example

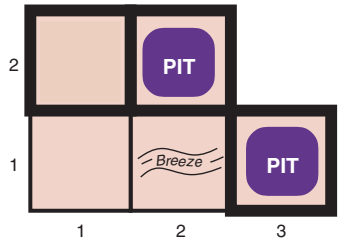
- The KB is true in models that agree with the agent's knowledge, false otherwise, e.g., it is false in model below, as there is no breeze in [1,1]




- There are only three models in which the KB is true:



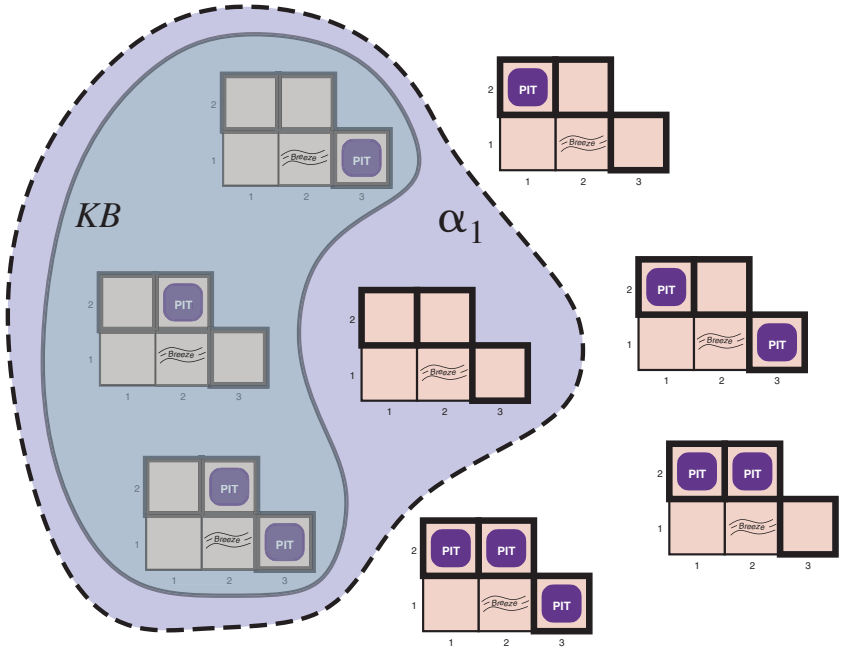



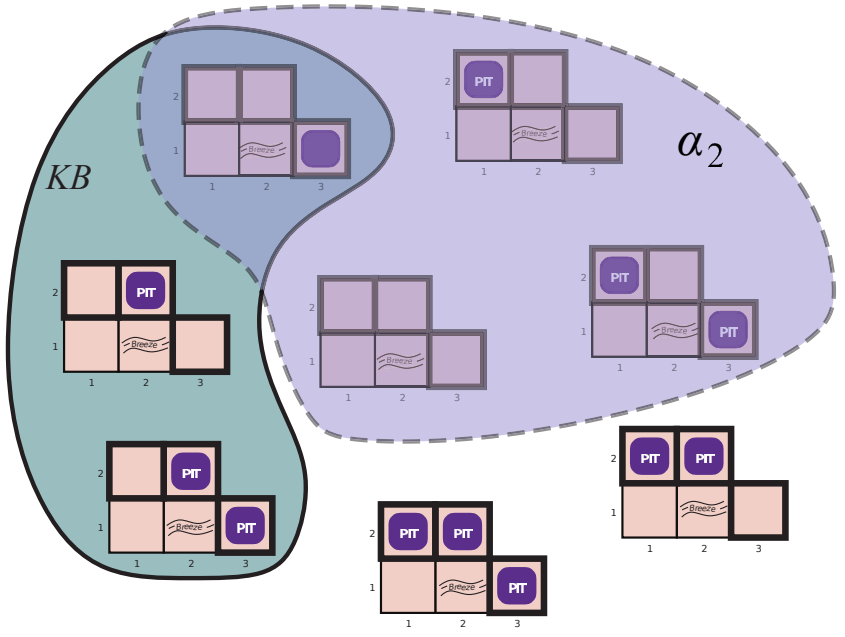



# Example

Two possible conclusions, and the respective models

- $\alpha_1$ = "there is no pit in [1,2]"
- $\alpha_2$ = "there is no pit in [2,2]"



# Example

- It can be seen that
  - in **every** model in which $KB$ is **true**, $\alpha_1$ is also **true**
  - in **some** model in which $KB$ is **true**, $\alpha_2$ is **false**
- By definition, this means that
  - $KB \models \alpha_1$ (the KB entails there is no pit in [1,2])
  - $KB \not\models \alpha_2$ (the KB does not entail there is no pit in [2,2])
- Therefore the agent
  - can conclude there is no pit in [1,2]
  - **cannot** conclude there is no pit in [2,2] (nor that there **is** a pit in [2,2])

# Logical inference: model checking

- The previous example shows how the definition of entailment, $KB \models \alpha$, can be applied to carry out logical inference
  1. enumerate all possible models of $KB$ and $\alpha$
  2. check whether $\alpha$ is true in **all** models in which $KB$ is true, i.e., whether $M(KB) \subseteq M(\alpha)$
- This inference algorithm is called model checking

# Properties of inference algorithms

- In general, the fact that an inference algorithm $A$ derives a sentence $\alpha$ from $KB$ is represented as

$$KB \vdash_A \alpha$$

- Two desirable properties
  - soundness (truth-preservation): $A$ derives **only** entailed sentences

$$\text{if } KB \vdash_A \alpha \text{ then } KB \models \alpha$$

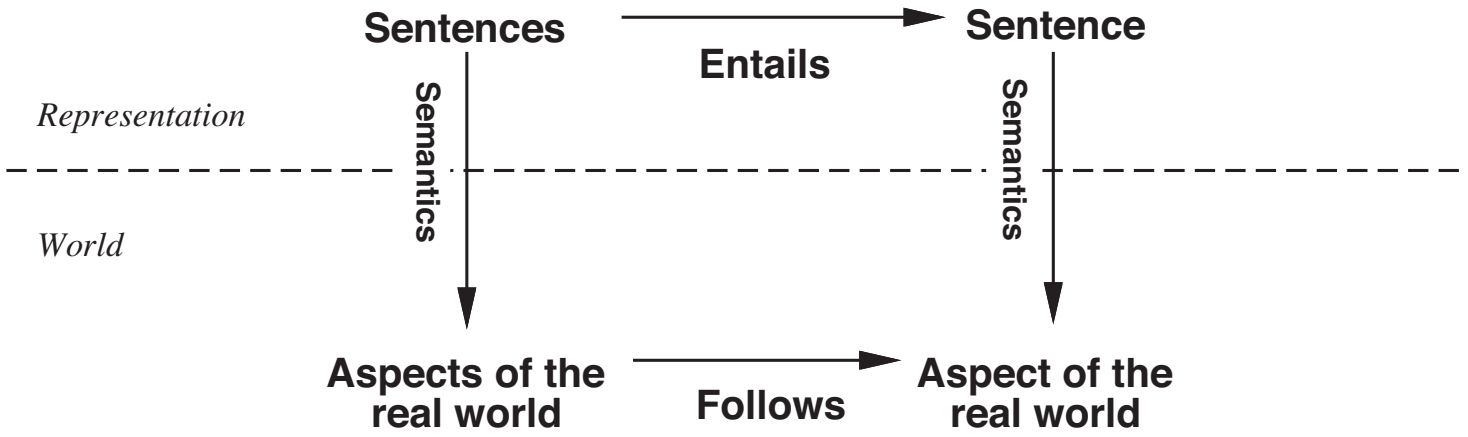
  - completeness: $A$ can derive **all** the entailed sentences

$$\text{if } KB \models \alpha \text{ then } KB \vdash_A \alpha$$

- By construction, model checking is
  - complete (provided that the set of models is **finite**)
  - sound

# Properties of inference algorithms

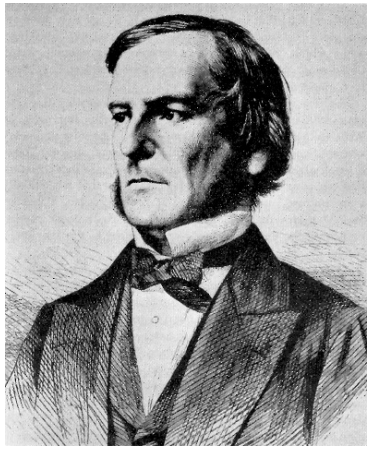
- Inference algorithms operate at the **syntactic** level: they just manipulate symbols
- Soundness guarantees that
  - if a KB is true in the **real** world
  - then any sentence $\alpha$ derived from that KB is also true in the real world



# Propositional logic

# Propositional logic

- The simplest logical language
- Extension of Boolean algebra
  - George Boole (1815–1864)
  - originally devised for representing propositions of natural language and modelling inference procedures
- Several notations exist



# Syntax: atomic sentences

- Atomic sentences: a single proposition symbol, representing a natural language proposition that can be either true or false
  - e.g., "there is a pit in room [1,2]"
  - proposition symbols start with an **uppercase** letter
  - examples: $P$, $Q$, $A_{1,2}$, $DeadBattery$
  - they are analogous to Boolean variables
- Two predefined symbols with constant meaning: $True$ and $False$
  - $True$ is an always-true proposition
  - $False$ is an always-false proposition
  - they are analogous to symbols $1$ and $0$ of Boolean logic

# Syntax: complex sentences

- Complex sentences are constructed from simpler ones using
  - logical connectives
  - parentheses
- Five commonly used connectives ($\alpha$ and $\beta$ denote any sentences)
  - $\neg$ (**not**): $\neg\alpha$ is the **negation** of $\alpha$
    literal: an atomic sentence (e.g., $P$) or a negated atomic sentence (e.g., $\neg P$)
  - $\wedge$ (**and**): $\alpha \wedge \beta$ is a **conjunction**
  - $\vee$ (**or**): $\alpha \vee \beta$ is a **disjunction** (**inclusive** OR, not exclusive OR)
  - $\Rightarrow$ (**implies**): $\alpha \Rightarrow \beta$ is an **implication**, $\alpha$ is the **premise** (**antecedent**) and $\beta$ the **conclusion** (**consequent**); also named **rule** of **if–then** statement
  - $\Leftrightarrow$ (**if and only if**): $\alpha \Leftrightarrow \beta$ is a **biconditional**

# Syntax in Backus–Naur Form

$$
\begin{aligned}
Sentence &\rightarrow AtomicSentence \mid ComplexSentence \\
AtomicSentence &\rightarrow True \mid False \mid Symbol \\
Symbol &\rightarrow P \mid Q \mid R \mid \ldots \\
ComplexSentence &\rightarrow \neg Sentence \\
&\mid (Sentence \wedge Sentence) \\
&\mid (Sentence \vee Sentence) \\
&\mid (Sentence \Rightarrow Sentence) \\
&\mid (Sentence \Leftrightarrow Sentence)
\end{aligned}
$$

operator precedence : $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

# Semantics

- Rules for determining the truth value of sentences in a given **model**
- In propositional logic a defining a model corresponds to setting the **truth value** of each **proposition symbol** (atomic sentence)
  - example: if $A$ represents the proposition "there is a pit in room [1,2]," it is true in wumpus world instances where room [1,2] contains a pit, false otherwise
- **Atomic** sentences
  - $True$ is true in every model, $False$ if false in every model
  - the truth value of every other proposition symbol is specified by the model
- **Complex** sentences
  - their truth value is determined **recursively** as a function of **logical connectives**
  - the semantics of logical connectives can be expressed with **truth tables**

# Semantics: truth tables

| $\alpha$ | $\beta$ | $\neg\alpha$ | $\alpha \wedge \beta$ | $\alpha \vee \beta$ | $\alpha \Rightarrow \beta$ | $\alpha \Leftrightarrow \beta$ |
|---|---|---|---|---|---|---|
| false | false | true | false | false | true | true |
| false | true | true | false | true | true | false |
| true | false | false | false | true | false | false |
| true | true | false | true | true | true | true |

Example

- a possible model for proposition symbols $A$, $B$ and $C$:
$$m = \{A = \text{false}, B = \text{false}, C = \text{true}\}$$
- the sentence $\neg A \wedge (B \vee C)$ evaluated in $m$ gives:
$$\text{true} \wedge (\text{false} \vee \text{true}) = \text{true} \wedge \text{true} = \text{true}$$

# Semantics of the implication connective

- The truth table of ⇒ may be not intuitive...
  - "five is odd implies Tokyo is the capital of Japan" is **true** according to the truth table...
  - "five is even implies Sam is smart" is **true** as well, regardless of whether Sam is smart or not...

| $\alpha$ | $\beta$ | $\alpha \Rightarrow \beta$ |
|---|---|---|
| false | false | true |
| false | true | true |
| true | false | false |
| true | true | true |

- The actual meaning of ⇒
  - it does **not** require **cause-effect** nor **relevance** relations between $\alpha$ and $\beta$
  - its can be understood as saying:
    "if $\alpha$ is true I am claiming $\beta$ is true as well, otherwise I am making **no** claim"
  - more formally, it expresses the concept of **sufficient condition**:
    "$\alpha$ being true is a sufficient condition for $\beta$ to be true"
    (thus, $\alpha$ being false does not set any constraint on the truth value of $\beta$)

# Exercise

- Define a set of proposition symbols to represent the KB of the wumpus world, focusing on
  - location of the agent
  - locations of pits
  - location of the wumpus
  - location of the breezes
  - location of the stenches
- Write sentences that represent the instance of the wumpus world on the right, focusing only on pits and breezes in rooms [1,1], [1,2], [2,1], [2,2] and [3,1]

| | | | |
|---|---|---|---|
| 1,4 | 2,4 | 3,4 | 4,4 |
| 1,3 | 2,3 | 3,3 | 4,3 |
| 1,2 OK | 2,2 P? | 3,2 | 4,2 |
| 1,1 V OK | 2,1 A B OK | 3,1 P? | 4,1 |

# A possible solution

- Propositional symbols:
  - $P_{x,y}$: "there is a pit in $[x, y]$", $B_{x,y}$: "there is a breeze in $[x, y]$", etc.
- Partial KB
  - **rule of the game**: there is no pit in [1,1]
    $$R_1: \neg P_{1,1}$$
  - **rule of the game**: a square is breezy if and only if there is a pit in a neighbouring room
    $$R_2: B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$
    $$R_3: B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$$
  - **percepts**: no breeze in [1,1], breeze in [2,1]
    $$R_4: \neg B_{1,1}$$
    $$R_5: B_{2,1}$$

| | | | |
|---|---|---|---|
| 1,4 | 2,4 | 3,4 | 4,4 |
| 1,3 | 2,3 | 3,3 | 4,3 |
| 1,2 OK | 2,2 P? | 3,2 | 4,2 |
| 1,1 V OK | 2,1 A B OK | 3,1 P? | 4,1 |

# Inference in propositional logic: example

- Does the previous KB entail the absence of pits in room [1,2]?
- Formally: determining whether $KB \models \alpha$, with
  - $KB = \{\neg P_{1,1},\ B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}),\ B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1}),\ \neg B_{1,1}, B_{2,1}\}$
  - $\alpha = \neg P_{1,2}$
- **Model checking** can be applied
  - 7 proposition symbols, $2^7 = 128$ possible models
  - construct the corresponding truth table...
  - ... and check whether in **all** the rows in which $KB$ is **true**, also $\neg P_{1,2}$ is **true**

# Inference in propositional logic: example

- It turns out there are only three rows in which KB is true
- It is easy to see that in all such rows $\neg P_{1,2}$ is true
- Therefore $KB \models \neg P_{1,2}$ ($\neg P_{1,2}$ is entailed by $KB$)

| $B_{1,1}$ | $B_{2,1}$ | $P_{1,1}$ | $P_{1,2}$ | $P_{2,1}$ | $P_{2,2}$ | $P_{3,1}$ | $R_1$ | $R_2$ | $R_3$ | $R_4$ | $R_5$ | $KB$ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| *false* | *false* | *false* | *false* | *false* | *false* | *false* | *true* | *true* | *true* | *true* | *false* | *false* |
| *false* | *false* | *false* | *false* | *false* | *false* | *true* | *true* | *true* | *false* | *true* | *false* | *false* |
| ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ |
| *false* | *true* | *false* | *false* | *false* | *false* | *false* | *true* | *true* | *false* | *true* | *true* | *false* |
| *false* | *true* | *false* | *false* | *false* | *false* | *true* | *true* | *true* | *true* | *true* | *true* | *true* |
| *false* | *true* | *false* | *false* | *false* | *true* | *false* | *true* | *true* | *true* | *true* | *true* | *true* |
| *false* | *true* | *false* | *false* | *false* | *true* | *true* | *true* | *true* | *true* | *true* | *true* | *true* |
| *false* | *true* | *false* | *false* | *true* | *false* | *false* | *true* | *false* | *false* | *true* | *true* | *false* |
| ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ |
| *true* | *true* | *true* | *true* | *true* | *true* | *true* | *false* | *true* | *true* | *false* | *true* | *false* |

# Model checking: computational complexity

- For a KB with $n$ proposition symbols there are $2^n$ possible **models**...
  - model checking is sound and complete...
  - ...but its time complexity is $\mathcal{O}(2^n)$
- In general, all **known** inference algorithms for propositional logic have worst-case complexity **exponential** in the size of the input
- Much more efficient algorithms exist for particular cases

# Inference in propositional logic

# Inference by theorem proving

- Checking whether $KB \models \alpha$ by theorem proving
  - applying **inference rules** directly to sentences of $KB$ to derive a **proof**, i.e., a chain of intermediate conclusions that leads to $\alpha$
  - this does **not** require to enumerate models
- The following concepts related to entailment are useful
  - logical equivalence
  - validity
  - satisfiability

# Logical equivalence

- Two sentences $\alpha$ and $\beta$ are logically equivalent ($\alpha \equiv \beta$) if they are true in the **same** set of models (e.g., $P \wedge Q \equiv Q \wedge P$)
- Useful logical equivalences:

$$
\begin{aligned}
(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\
(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\
((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\
((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\
\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\
(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\
\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{De Morgan} \\
\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{De Morgan} \\
(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\
(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge
\end{aligned}
$$

# Validity

- A sentence is valid if it is true in **all** models
  - example: $P \vee \neg P$ is valid, $P \vee Q$ is not
- Valid sentences are also known as **tautologies** (necessarily true)
- Connection with **entailment** (**deduction theorem**):

$$\alpha \vDash \beta \quad \text{if and only if} \quad (\alpha \Rightarrow \beta) \text{ is valid}$$

# Satisfiability

- A sentence $\alpha$ is satisfiable if it is true in **at least one** model (e.g.: $P \wedge Q$ is satisfiable, $P \wedge \neg P$ is not)
- Connection with **validity**:

  $\alpha$ is valid if and only if $\neg\alpha$ is unsatisfiable
- Connection with **entailment**:

  $\alpha \vDash \beta$ if and only if $(\alpha \wedge \neg\beta)$ is unsatisfiable
- The latter result is used in the mathematical technique of proof by refutation or by contradiction (*reductio ad absurdum*)
  - assuming $\beta$ is false
  - determining whether this leads to a contradiction: if so, then $\alpha \vDash \beta$

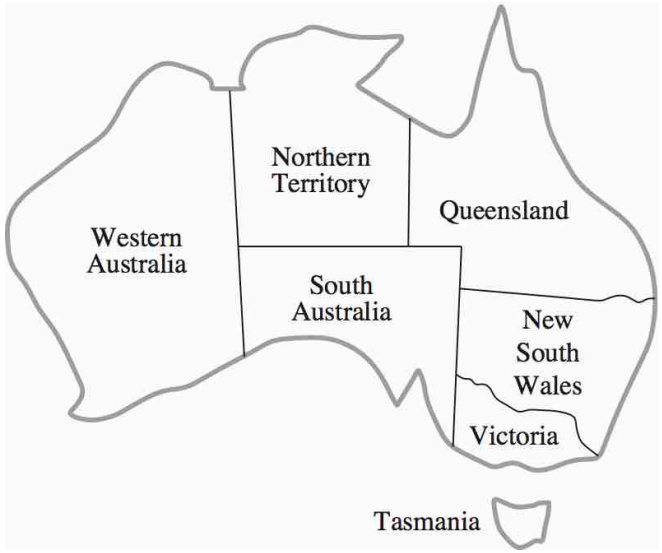
# Propositional Satisfiability (SAT) Problem

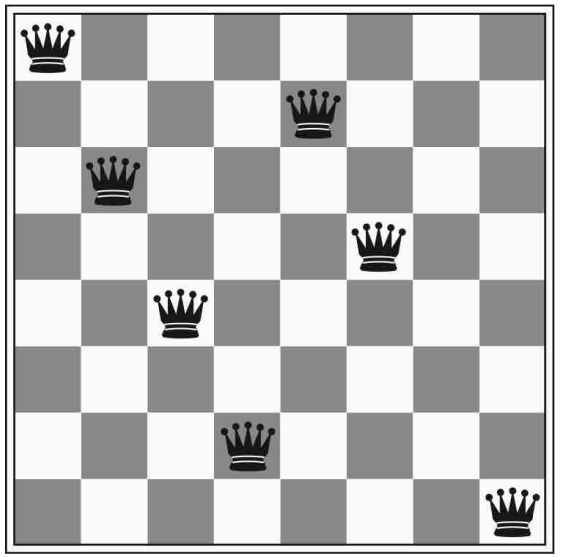
- Determining whether a set of sentences in propositional logic is satisfiable: a well-known problem in logic
- The SAT problem is NP-complete
  - worst-case time complexity: **exponential** with the instance size
  - many other problems can be proven to be NP-complete via a reduction to SAT
- Applications
  - many automated reasoning problems in propositional logic are formulated and solved as SAT problems
  - so are problems in many other areas: software analysis and verification, integrated circuit design and verification, planning, computer-aided design (CAD) and manufacturing, scheduling, etc.

# SAT problems: examples

Constraint satisfaction problems can be formlated as SAT problems

- map colouring
- 8-queens
- ...



# Reasoning patterns: inference rules

- Notation: $\frac{\alpha_1, \ldots, \alpha_n}{\beta}$
- Meaning: whenever sentences $\alpha_1, \ldots, \alpha_n$ are **given**, $\beta$ can be **inferred**
- **Soundness** can be easily proven by model checking (the number of sentences involved is always very small)
- All previous logical equivalences can be used as inference rules, e.g., **biconditional elimination** leads to

$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)} \quad \text{and} \quad \frac{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta}$$

# Two useful inference rules

- Modus Ponens (a very well known inference rule):

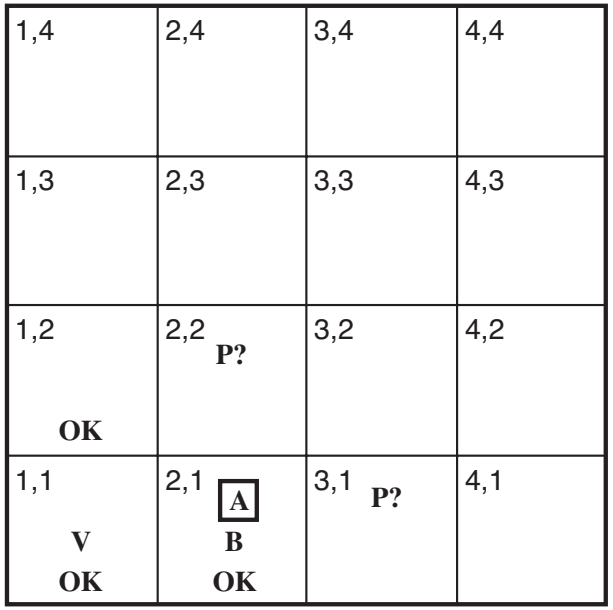
$$\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$$

- And elimination:

$$\frac{\alpha \wedge \beta}{\alpha}$$

# Theorem proving: example

- The same KB of the previous example on model checking:
  - $R_1: \neg P_{1,1}$
  - $R_2: B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
  - $R_3: B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$
  - $R_4: \neg B_{1,1}$
  - $R_5: B_{2,1}$
- Goal: proving whether $KB \models \neg P_{1,2}$ using inference rules



# Theorem proving: example

1. Biconditional elimination to $R_2$:

   $R_6: \left(B_{1,1} \Rightarrow \left(P_{1,2} \vee P_{2,1}\right)\right) \wedge$

   $\left(\left(P_{1,2} \vee P_{2,1}\right) \Rightarrow B_{1,1}\right)$

2. And-elimination to $R_6$:

   $R_7: \left(P_{1,2} \vee P_{2,1}\right) \Rightarrow B_{1,1}$

3. Contraposition to $R_7$:

   $R_8: \neg B_{1,1} \Rightarrow \neg\left(P_{1,2} \vee P_{2,1}\right)$

4. Modus Ponens to $R_8$ and $R_4$:

   $R_9: \neg\left(P_{1,2} \vee P_{2,1}\right)$

5. De Morgan's rule to $R_9$:

   $R_{10}: \neg P_{1,2} \wedge \neg P_{2,1}$

6. And-elimination to $R_{10}$:

   $R_{11}: \neg P_{1,2}$

This proves that $KB \models \neg P_{1,2}$

# Monotonicity

- In the previous proof
  - only part of the whole KB of the wumpus world has been used to derive $\neg P_{1,2}$
  - newly derived sentences have been used in subsequent derivations
- This is possible since logical systems are monotonic
  - the set of entailed sentences can only **increase** as information is added to a KB (provided that the whole KB is **not contradictory**)
  - formally, for any sentences $\alpha$ and $\beta$: if $KB \models \alpha$ then $KB \wedge \beta \models \alpha$
- Additional knowledge could allow deriving **additional** conclusions, but it cannot invalidate already inferred ones

# Efficiency of theorem proving

- **Which** inference rule to **which** sentences, at each step?
- Many possible choices, leading to a high computational complexity (this process is analogous to a **search problem**)
- Often it is possible to ignore **irrelevant** propositions, making theorem proving **more efficient** than model checking
- Example: in the previous proof propositions like $B_{2,1}$, $P_{1,1}$, $P_{2,2}$ have been disregarded, because
  - the goal proposition $P_{1,2}$ appears only in sentence $R_2$
  - the other propositions in $R_2$ appear only in sentence $R_4$
  - …

# Completeness of theorem proving

- Inference rules provide **sound** derivations
- Are algorithms that use them also **complete**?
  - remember: an inference algorithm $A$ is complete if it can derive all the sentences entailed by a KB, i.e.: if $KB \models \alpha$ then $KB \vdash_A \alpha$
- Theorem proving can be seen as a **search problem**
  - goal: proving whether $KB \models \alpha$
  - start state: the initial KB
  - operators: a given set of inference rules
  - state space: the set of all possible KBs that can be obtained by applying any possible sequence of inference rules to the initial KB
  - goal states: every state (KB) that contains $\alpha$
- Is it guaranteed that, if $KB \models \alpha$, then at least one goal state is present in the state space? If not, $A$ would be **not complete**
- (By the way, what is the **computational complexity** of this search problem?)

# The resolution inference algorithm

- Fortunately, a **complete** inference algorithm for propositional logic is known: resolution
- It is based on the **single**, homonymous resolution inference rule
- The resolution rule can be applied only to sentences in Conjunctive Normal Form (CNF)
  - a **conjunction** of clauses
  - clause: a **disjunctions** of **literals**
  - examples: $P, \quad \neg Q, \quad P \vee \neg Q, \quad P \vee \neg Q \vee R, \quad (P \vee \neg Q \vee R) \wedge (T \vee \neg R)$
- This is **not** a limitation: **every** propositional sentence can be rewritten in CNF

# Conjunctive Normal Form

The following procedure allows every sentence to be rewritten in CNF

1. biconditional elimination: $(\alpha \Leftrightarrow \beta) \equiv \big((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)\big)$
2. implication elimination: $(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$
3. De Morgan's laws, to "move negations inwards" up to individual proposition symbols
   - $\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$
   - $\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$
4. double-negation elimination: $\big(\neg(\neg\alpha)\big) \equiv \alpha$
5. distributing $\vee$ over $\wedge$: $(\alpha \wedge \beta) \vee \gamma \equiv (\alpha \vee \gamma) \wedge (\beta \vee \gamma)$

# Example

Consider sentence $R_2$ of the previous KB: $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$

1. biconditional elimination:
   $(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$
2. implication elimination:
   $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$
3. De Morgan's law:
   $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$
4. double-negation elimination: not applicable
5. distributing $\vee$ over $\wedge$:
   $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$

# Knowledge bases in CNF

- By And-elimination, a KB in CNF is equivalent to the set of the corresponding clauses (they entail the **same** set of sentences)
- Example: the following KBs are equivalent
  - $\{(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})\}$
  - $\{(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}), \quad (\neg P_{1,2} \vee B_{1,1}), \quad (\neg P_{2,1} \vee B_{1,1})\}$
- Every KB in CNF can thus be rewritten as the set of its **clauses**

# The resolution inference rule

- Given any two disjunctions of literals
  - $\alpha = \ell'_1 \vee \cdots \vee \ell'_p$ and $\beta = \ell''_1 \vee \cdots \vee \ell''_q$
  - where $\ell'_i$ in $\alpha$ and $\ell''_j$ in $\beta$ are **complementary** literals (e.g.: $\ell'_i = P$, $\ell''_j = \neg P$)

$$\frac{\ell'_1 \vee \cdots \vee \ell'_p, \qquad \ell''_1 \vee \cdots \vee \ell''_q}{\ell'_1 \vee \cdots \ell'_{i-1} \vee \ell'_{i+1} \vee \cdots \vee \ell'_p \vee \ell''_1 \vee \cdots \ell''_{j-1} \vee \ell''_{j+1} \vee \cdots \vee \ell''_q \vee \ell''_q}$$

- The derived clause is the disjunction of **all** the literals in $\alpha$ and $\beta$, **except** for the two complementary literals $\ell'_i$ and $\ell''_j$
- Soundness can be easily proven using truth tables (model checking)
- Intuitive explanation: if $\gamma \vee P$ and $\delta \vee \neg P$ are both true, then either $\gamma$ or $\delta$ (or both) must be true, i.e., $\gamma \vee \delta$ must be true

# Example

- Two sentences in CNF (clauses) related to the wumpus world
  - $P_{1,1} \vee P_{3,1}$
  - $\neg P_{1,1} \vee \neg P_{2,2}$
- They contain two complementary literals: $P_{1,1}$ and $\neg P_{1,1}$
- Resolving them yields

$$\frac{P_{1,1} \vee P_{3,1}, \quad \neg P_{1,1} \vee \neg P_{2,2}}{P_{3,1} \vee \neg P_{2,2}}$$

# Resolution rule: particular cases

- Duplicated literals can be reduced to one, e.g.:

$$\frac{P \vee Q, \quad P \vee \neg Q \vee \neg R}{P \vee P \vee \neg R} \rightarrow \quad P \vee \neg R$$

  - in general, $\alpha \vee \alpha \equiv \alpha$

- Only **one** pair of literals at a time can be resolved, e.g.:

$$\frac{P \vee \neg Q \vee R, \quad \neg P \vee Q}{\neg Q \vee Q \vee R}$$

  - any clause containing coplementary literals is trivially **true**, thus **useless** in derivations: it can be **discarded**

- Complementary literals are **contradictory**: their resolution yelds an **empty** clause, represented by [ ]: $\frac{P, \quad \neg P}{[\ ]}$
  - a KB should **not** be contradictory...

# Theorem proving using resolution

- Inference algorithms based on the resolution inference rule adopt the **proof by refutation** technique:

$$KB \models \alpha \quad \text{if and only if} \quad (KB \wedge \neg\alpha) \text{ is } \textbf{unsatisfiable}$$

- To prove whether $KB \models \alpha$, check whether $KB \wedge \neg\alpha$ is satisfiable
  - if it is, then one can conclude that $KB \models \alpha$
  - if it is not, then one can conclude that $KB \not\models \alpha$

# A resolution inference algorithm

- Informal description
  - convert $KB \wedge \neg\alpha$ into CNF
  - apply resolution in **all** possible ways to pairs of clauses of $KB$, and **add** the resulting clauses to $KB$ if not already present
  - repeat the previous step until one of two thigs happen
    - no **new** clauses have been derived: this means $KB \not\models \alpha$
    - two **complementary** literals are resolved, yielding [ ]: this means $KB \models \alpha$
- This procedure is guaranteed to end after a **finite** number of steps
- This inference algorithm is **complete**

# A resolution inference algorithm

Formally:

**function** RESOLUTION ($KB$, $\alpha$) **returns** *true* or *false*
  *clauses* $\leftarrow$ the set of clauses of $KB \cup \{\neg\alpha\}$ in CNF form
  *new* $\leftarrow \emptyset$
  **loop do**
    **for each** distinct $\alpha_i, \alpha_j$ **in** *clauses* **do**
      *resolvents* $\leftarrow$ `resolve`$(\alpha_i, \alpha_j)$
      **if** *resolvents* contains [ ] **then return** *true*
      *new* $\leftarrow$ *new* $\cup$ *resolvents*
    **if** *new* $\subseteq$ *clauses* **then return** *false*
    *clauses* $\leftarrow$ *clauses* $\cup$ *new*

`resolve` applies the resolution rule to **all** pairs of complementary literals found in **all** pairs of clauses $(\alpha_i, \alpha_j)$ of the current KB

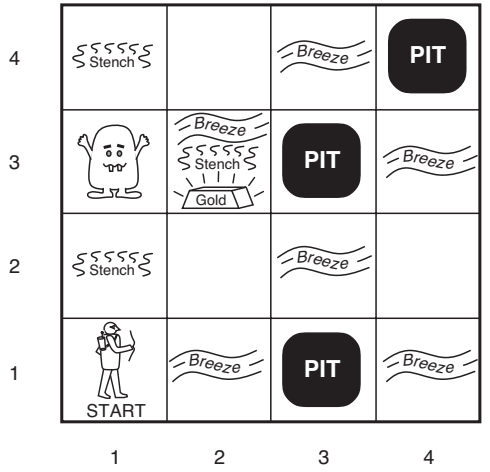
# Example

- A possible initial state of the wumpus world
  - the agent is in room [1,1]
  - it does not perceive any breeze
  - it is interested in whether room [1,2] does not contain any pit: $\alpha = \neg P_{1,2}$
  - a relevant part of the KB is:

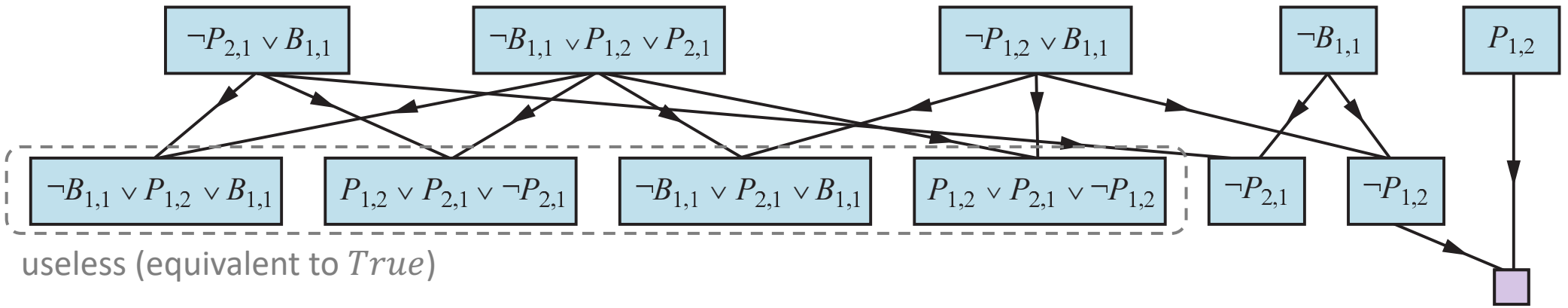
$$KB = \{\neg B_{1,1}, B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})\}$$

  - after rewriting $KB \wedge \neg\alpha$ in CNF (see the previous example):

$$KB = \{\neg B_{1,1}, (\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}), (\neg P_{1,2} \vee B_{1,1}), (\neg P_{2,1} \vee B_{1,1}), P_{1,2}\}$$



# Example




- first row: $KB \wedge \neg\alpha$
- second row: distinct clauses derived by resolving all possible pairs of complementary literals in the first row (four of them are **useless**)
- the empty caluse is obtained when resolving $\neg P_{1,2}$ and $P_{1,2}$: this means that the agent's KB entails $\alpha$ (there is no pit in room [1,2])

# Definite clauses and Horn clauses

- In several practical problems the KB is made up of restricted kinds of clauses, which allow to use more efficient inference algorithms
- Definite clause: disjunction of literals of which **exactly one** is positive
  - example: $\neg S_{2,1} \vee \neg S_{4,1} \vee \neg S_{3,2} \vee W_{3,1}$
  - logically equivalent to an implication with a conjunction of positive literals as the antecedent (called body) and a single positive literal as the consequent (called head), e.g.: $\neg S_{2,1} \vee \neg S_{4,1} \vee \neg S_{3,2} \vee W_{3,1} \equiv \left(S_{2,1} \wedge S_{4,1} \wedge S_{3,2}\right) \Rightarrow W_{3,1}$
  - they can be interpreted as **if-then rules**
  - a definite clause consisting of a single positive literal (e.g., $B_{1,1}$) is called a fact
- Horn clause: disjunction of literals of which **at most one** is positive

# Forward and Backward chaining

- If a KB is made up of **definite clauses**, the **Modus Ponens** inference rule allows to define inference algorithms capable of deriving **all** the entailed facts (positive literals, i.e., atomic sentences)